Figure 1.4 The front controller used in MVC coordinates processing for multiple views and can get very busy and complex. In Razor Pages, each page has its own controller, keeping them lean and easier to work with.

1.3.3 The design goals of Razor Pages

As you've already learned, the MVC framework is an opinionated framework. If you want to use it, you need to work with the framework authors' conventions or develop some kind of workaround. ASP.NET MVC includes a number of conventions for naming files and placing them within the application. For example, suppose your customer or boss wants you to add a new feature to an existing MVC application. Bearing in mind that front controller classes are feature-driven by convention, you would have to add a new class file representing the feature to your Models folder, a new controller class to the Controllers folder, a new folder for the feature in the Views folder, a new Razor view to that folder, and, finally, a `viewmodel` class to represent the data for your view. If you want to make any changes to the feature, you have to dip in and out of folders and files all over the code base.

Developers who are new to the MVC pattern can find the complexity of working with the ASP.NET implementation quite daunting. If you are not familiar with the structure of an ASP.NET MVC application and found yourself getting a bit lost with the workflow I just described, welcome to my target audience! Even Microsoft themselves have described the framework as having a "high concept count." So the design goals of Razor Pages (https://github.com/aspnet/mvc/issues/494) are set against that backdrop and implicitly compare working with Razor Pages to the MVC framework. They include (to quote from the referenced GitHub issue) the following: